

Compte-rendu

Traitement statistique des données

1. Introduction.....	1
1.1. Objectif.....	1
1.2. Données et Ressources.....	1
1.2.1. Présentation du corpus.....	1
1.2.2. Prétraitement du corpus.....	2
2. Méthodologie.....	3
2.1. La distinction entre les deux types de textes.....	3
2.2. La comparaison des classificateurs.....	3
2.3. Mise en œuvre.....	4
3. Expériences réalisées.....	5
4. Résultats & Analyses.....	6
4.1. Résultats initiaux.....	6
4.2. Optimisation par les n-grammes de POS.....	9
5. Bilan.....	10
6. Références.....	12
7. Annexes.....	13

Vous trouverez le dépôt GitHub de notre travail via ce [lien](#)

L'ensemble du projet a été réalisé conjointement par les deux membres du groupe,
lors de séances de travail en présentiel.

1. Introduction

1.1. Objectif

L'objectif de ce projet est d'*entraîner des classificateurs par apprentissage automatique et de comparer les performances de différents algorithmes de classification sur une tâche* (consigne du projet) choisie par notre groupe.

Pour notre projet, nous avons choisi l'identification automatique de textes écrits par les locuteurs natifs et les apprenants du français comme notre tâche. Plus précisément, nous voulons voir comment des classificateurs peuvent distinguer les productions de francophones natifs et celles d'apprenants de français à partir des catégories morphosyntaxiques des mots utilisées dans les textes.

À partir de plusieurs expériences comparatives réalisées avec [Weka](#)², nous cherchons donc à analyser les résultats obtenus avec trois classificateurs : J48, Naive Bayes et SVM.

1.2. Données et Ressources

1.2.1. Présentation du corpus

Voici la répartition des textes dans le corpus CEAAL2 selon les différentes catégories d'éditeur fournie par le [site](#)¹ :

Productions de groupes de contrôle

- Francophones natifs (essai argumenté) : 41
- Francophones natifs (lettre formelle) : 58
- Polonophones natifs (essai argumenté) : 10
- Russophones natifs (lettre formelle) : 37
- Sinophones natifs (lettre formelle) : 15
- Indonésiens natifs (lettre formelle) : 15

Productions d'apprenants

- Apprenants sinophones (contexte homoglote, essai argumenté) : 9
- Apprenants sinophones (contexte homoglote, lettre formelle, DSA, Licence, Master) : 35
- Apprenants sinophones (contexte homoglote, lettre formelle, DU LLFC) : 22
- Apprenants sinophones (contexte hétéroglote, lettre formelle) : 66
- Apprenants polonophones (contexte hétéroglote, lettre formelle, essai argumenté) : 73
- Apprenants russophones (contexte hétéroglote, lettre formelle) : 51
- Apprenants indonésiens (contexte hétéroglote, lettre formelle) : 17
- Groupes hétérogènes en contexte homoglote (lettre formelle, essai) : 74

Pour notre projet, on utilise le corpus *CEAAL2 (Corpus écrits d'apprenants et acquisition de L2)*

- Source : Il est disponible sur la plateforme [Ortolang](#)³. Il a été produit par le laboratoire LIDILEM de l'Université Grenoble Alpes.
- Type de corpus : C'est un corpus écrit composé de productions rédigées par deux types des éditeurs : les locuteurs natifs de différentes langues et les apprenants de français étrangers : chinois, polonais, russe, indonésien.

- Format : Il est au format **.pdf**, le corpus est organisé en plusieurs fichiers PDF selon le profil linguistique des éditeurs, par exemple les francophones natifs, les apprenants sinophones ou les apprenants russophones. Chaque fichier PDF contient plusieurs textes, et chaque texte est précédé de métadonnées sur le scripteur, comme l'âge, le sexe, le niveau d'étude ou les langues parlées.
- Droits d'utilisation : Libre sans utilisation commerciale.

Voici les fichiers utilisés pour notre corpus :

Natifs	Apprenants
- Francophones natifs (essai argumenté) : 41 - Francophones natifs (lettre formelle) : 58	- Apprenants sinophones (contexte hétéroglotte, lettre formelle) : 66 - Apprenants russophones (contexte hétéroglotte, lettre formelle) : 51

Pour la classe des natifs, nous avons seulement retenu les textes des francophones natifs, sans utiliser les productions de natifs d'autres langues. Nous avons aussi remarqué que les informations affichées sur *Ortolang* ne semblent pas totalement correspondre aux fichiers téléchargés. Même si le site indiquait 99 textes de francophones natifs, nous avons pu extraire 100 textes.

Pour la classe des apprenants, nous avons choisi des apprenants en contexte hétéroglotte, c'est-à-dire des apprenants qui ne sont pas en France, afin de rendre la distinction entre les deux classes plus nette et comme le nombre de textes disponibles dans les deux classes ne sont pas exactement le même, nous avons choisi d'équilibrer notre corpus, nous avons donc finalement conservé 100 textes de natifs et 100 textes d'apprenants.

1.2.2. Prétraitement du corpus

PDF → .txt → POS → .arff

idée principale du prétraitement

Après la constitution du corpus, nous avons d'abord extrait les textes contenus dans les fichiers **.pdf** et nous les avons enregistrés au format **.txt**, en conservant un fichier par production. Nous avons ensuite supprimé les métadonnées présentes avant chaque texte, ainsi que certains éléments qui ne faisaient pas partie de la production elle-même.

- Participant 1: **Naïs** (FR_L1_1)
- Langue maternelle : français
- Langue parlée avec parents, frères, sœurs, conjoint : français
- Age : 24
- Sexe : Féminin
- Niveau d'études : Licence 3 en cours
- Autres langues apprises : Anglais, espagnol, russe, arabe
- Séjours à l'étranger de longue durée (plus de 3 mois) : Non

Bien à vous.

Madame Cruchmule.

un exemple de métadonnées supprimées

un exemple d'éléments hors production supprimés

Comme les fichiers contenant les productions de francophones natifs et ceux contenant les productions d'apprenants ne présentaient pas exactement la même structure, nous avons

utilisé deux scripts distincts pour cette étape : [construire_corpus_natifs.py](#) et [construire_corpus_apprenants.py](#). Nous avons également réparti les textes en deux ensembles dans cette même étape : un corpus d'entraînement et un corpus de test, selon une proportion de 80 % / 20 %.

```

corpus
├── test
│   ├── apprenants [20 entries exceeds filelimit, not opening dir]
│   └── natifs [20 entries exceeds filelimit, not opening dir]
└── train
    ├── apprenants [80 entries exceeds filelimit, not opening dir]
    └── natifs [80 entries exceeds filelimit, not opening dir]

```

arborescence du corpus et nombre de fichiers

2. Méthodologie

Avant de réaliser les expériences de classification, nous avons dû définir deux choix méthodologiques principaux. Le premier concerne la nature des indices utilisés pour distinguer les productions de francophones natifs et celles d'apprenants de français : il ne s'agit pas seulement de laisser les classificateurs s'appuyer sur les mots des textes, mais de chercher une représentation plus abstraite des productions. Le second concerne la manière de comparer les classificateurs de façon cohérente : pour que les résultats soient interprétables.

2.1. La distinction entre les deux types de textes

Comme notre objectif n'est pas de voir si les classificateurs peuvent reconnaître les textes à partir de leurs thèmes ou de quelques mots caractéristiques, mais plutôt d'observer s'ils peuvent s'appuyer sur des différences liées à l'organisation linguistique des productions. Nous avons choisi de représenter les textes à partir de leurs catégories morphosyntaxiques qui permet de limiter l'influence du contenu lexical, des thèmes abordés ou de certains mots propres à un type de texte et mettre l'accent sur comme la répartition des noms, verbes, adjectifs, déterminants ou pronoms dans les productions écrites.

2.2. La comparaison des classificateurs

Nous avons J48(tree), NaiveBayes, SVM comme les classificateurs à comparer, et pour les comparer de façon plus complète, nous avons choisi de les tester sur les trois modes de représentation des attributs : les [occurrences](#), les valeurs [booléennes](#) et les [fréquences](#) fournies par le script [vectorisation.py](#) :

- occurrences : nombre d'apparitions d'une catégorie morphosyntaxique dans un texte
- booléenne : l'attribut indique seulement si une catégorie est présente ou absente
- fréquences : la proportion de chaque catégorie dans le document

En croisant les trois classifieurs avec ces trois modes de représentation, nous cherchons donc à observer si les performances dépendent plutôt du choix de l'algorithme, du type de représentation des attributs, ou de l'interaction entre les deux.

2.3. Mise en œuvre

Après avoir défini ces choix méthodologiques, nous avons obtenu le pipeline suivant pour préparer les données et réaliser les expériences :

PDF → TXT nettoyé → séquence POS → vectorisation .arff → weka

occurrences	J48
booléenne	NaiveBayes
fréquences	SVM

pipeline générale

Une fois les fichiers `.txt` constitués après le prétraitement, nous avons ensuite généré une nouvelle version du corpus composée uniquement de catégories morphosyntaxiques à l'aide du script [transformation_pos.py](#).

Partir étudier à l'étranger est une bonne chose

VERB VERB ADP NOUN VERB AUX DET ADJ NOUN

un exemple de transformation

Après cette transformation, nous avons utilisé le script [vectorisation.py](#) pour convertir le corpus POS au format `.arff`, afin de pouvoir l'exploiter dans Weka. Nous avons d'abord vectorisé le dossier `train` de `corpus_pos`, ce qui a permis de produire le fichier `train.arff`, ce fichier contient donc les représentations vectorielles des textes d'entraînement, à partir des étiquettes morphosyntaxiques extraites précédemment. Ensuite, nous avons vectorisé le dossier `test` en utilisant le lexique construit à partir du corpus d'entraînement. Pour la vectorisation, nous avons pris en compte trois modes de représentation des attributs : les occurrences, les valeurs booléennes et les fréquences.

```

arff
├── booléenne
│   ├── test_booléenne.arff
│   └── train_booléenne.arff
├── frequences
│   ├── test_frequences.arff
│   └── train_frequences.arff
└── occurrences
    ├── test_occurrences.arff
    └── train_occurrences.arff

```

arborescence du dossier arff

Cette organisation des fichiers nous a permis de lancer ensuite les expériences dans Weka de manière systématique. Chaque mode de représentation dispose de son propre fichier d'entraînement et de son propre fichier de test, qui seront utilisés pour comparer [J48](#), [Naive Bayes](#) et [SVM](#). Les paramètres et le déroulement précis des expériences sont présentés dans la partie suivante.

3. Expériences réalisées

Après la préparation des fichiers `.arff`, nous avons réalisé les expériences de classification avec Weka. Notre objectif était de comparer les performances de trois classifieurs, J48, Naive Bayes et SVM, sur les mêmes données et avec les mêmes conditions expérimentales. Pour chaque mode de représentation des attributs, nous avons utilisé une paire de fichiers : un fichier d'entraînement et un fichier de test. Par exemple, pour la représentation par occurrences, nous avons utilisé `train_occurrences.arff` pour entraîner les modèles et `test_occurrences.arff` pour les évaluer. La même procédure a été appliquée aux représentations booléennes et aux fréquences.

Mode de représentation	Fichier d'entraînement	Fichier de test
Occurrences	<code>train_occurrences.arff</code>	<code>test_occurrences.arff</code>
Booléenne	<code>train_booléenne.arff</code>	<code>test_booléenne.arff</code>
Fréquences	<code>train_frequences.arff</code>	<code>test_frequences.arff</code>

Pour chacun de ces trois modes, nous avons testé les trois classifieurs retenus : J48, Naive Bayes et SVM(SMO). Cela donne donc neuf expériences au total, et pour évaluer les performances, nous avons relevé les résultats fournis par Weka après chaque expérience :

```

resultats
├── booléenne
│   ├── J48.txt
│   ├── NaiveBayes.txt
│   └── SVM.txt
├── frequences
│   ├── J48.txt
│   ├── NaiveBayes.txt
│   └── SVM.txt
└── occurrences
    ├── J48.txt
    ├── NaiveBayes.txt
    └── SVM.txt
  
```

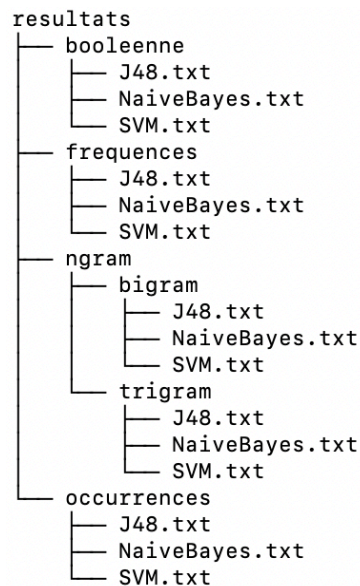
les résultats obtenus

Nous avons constaté que les premières expériences reposaient sur les catégories morphosyntaxiques prises séparément. Cette représentation permet de tenir compte de la présence, du nombre ou de la proportion des catégories, mais elle ne prend pas directement en compte leur ordre dans le texte. Pour résoudre cette limite et optimiser notre représentation des textes, nous avons donc ajouté une série d'expériences fondées sur les n-grammes de POS qui intègrent les enchaînements locaux de catégories morphosyntaxiques, par exemple "ADP + DET + NOUN", afin d'observer si ces séquences apportent des informations supplémentaires pour la classification.

Pour préparer cette expérience, nous avons utilisé le script [vectorisation_ngram.py](#), qui génère un fichier vectorisé à partir du corpus au format `.txt`. Ce script parcourt les fichiers du `corpus` extrait pour chaque production une séquence complète de catégories POS, puis associe cette séquence à sa classe, apprenants ou natifs. Contrairement aux premières

expériences, nous n'avons utilisé que le mode par occurrences pour les n-grammes. Les n-grammes génèrent en effet beaucoup plus d'attributs, ce qui rend les données plus dispersées. Les modes booléen et fréquentiel n'ont pas donné de résultats exploitables, et la séparation entre `train` et `test` a entraîné des problèmes de compatibilité des attributs. Nous avons donc travaillé à partir d'un fichier unique `total_ngram.arff`.

À partir de ce fichier unique, nous avons ensuite réalisé les expériences dans Weka avec les trois mêmes classifieurs : `J48`, `Naive Bayes` et `SVM(SMO)`. Pour cette partie, nous avons testé deux tailles de n-grammes : les `bigrammes` et les `trigrammes` de POS :



les résultats obtenus avec l'optimisation

4. Résultats & Analyses

Dans cette partie, nous analysons les résultats à partir de trois indicateurs d'évaluation : l'accuracy, le coefficient Kappa et la F-mesure pondérée. Les résultats complets fournis par Weka pour chaque expérience sont disponibles [ici](#).

4.1. Résultats initiaux

Dans cette première série de résultats, nous analysons les expériences selon deux axes : d'abord la comparaison des trois modes de représentation des données, puis la comparaison des trois classifieurs.

4.1.1. Comparaison de la représentation des données

Representation	Algorithmes	Accuracy %	Kappa	F-Measure
Booléenne	Naive Bayes	55	0.1	0.55
	SVM (SMO)	50	0	0.495
	J48	50	0	0.495
Occurrences	Naive Bayes	67.5	0.35	0.673
	SVM (SMO)	60	0.2	0.596
	J48	62.5	0.25	0.623
Fréquences	Naive Bayes	60	0.2	0.583
	SVM (SMO)	62.5	0.25	0.623
	J48	75	0.5	0.749

Performances des classifieurs selon les représentations

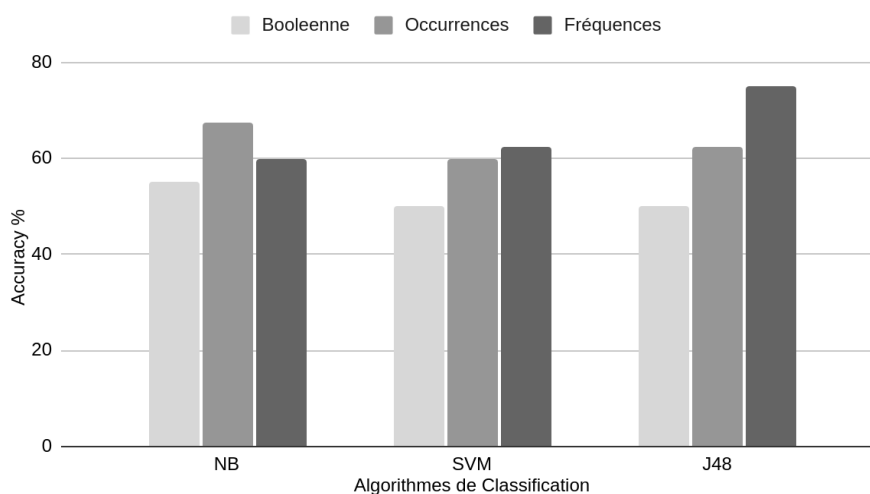
D'après les résultats, la représentation booléenne donne les scores les plus bas. Les trois accuracy se situent entre 50 % et 55 %, et les valeurs de Kappa sont très faibles.

Pour la représentation par occurrences, les scores sont plus élevés. Les trois classifieurs obtiennent tous une accuracy supérieure ou égale à 60 %, avec des valeurs de Kappa comprises entre 0,2 et 0,35.

Pour la représentation par fréquences, les résultats sont plus contrastés. Deux classifieurs restent autour de 60 %–62,5 %, tandis qu'un résultat atteint 75 %, avec un Kappa de 0,5 et une F-mesure de 0,749.

Ces observations montrent que la représentation booléenne est la moins adaptée à notre tâche, tandis que les représentations quantitatives donnent de meilleurs résultats. Les occurrences semblent offrir des performances plus régulières selon les classifieurs, alors que les fréquences permettent d'obtenir le meilleur score global, mais avec des écarts plus marqués entre les algorithmes.

4.1.2. Comparaison des classifieurs



Graphique à barres de performance des classifieurs

En visualisant les résultats par le graphique, nous pouvons constater que Naive Bayes obtient des résultats différents selon le mode de représentation. Son score le plus élevé apparaît avec les occurrences, tandis que ses résultats sont plus bas avec les fréquences et les valeurs booléennes.

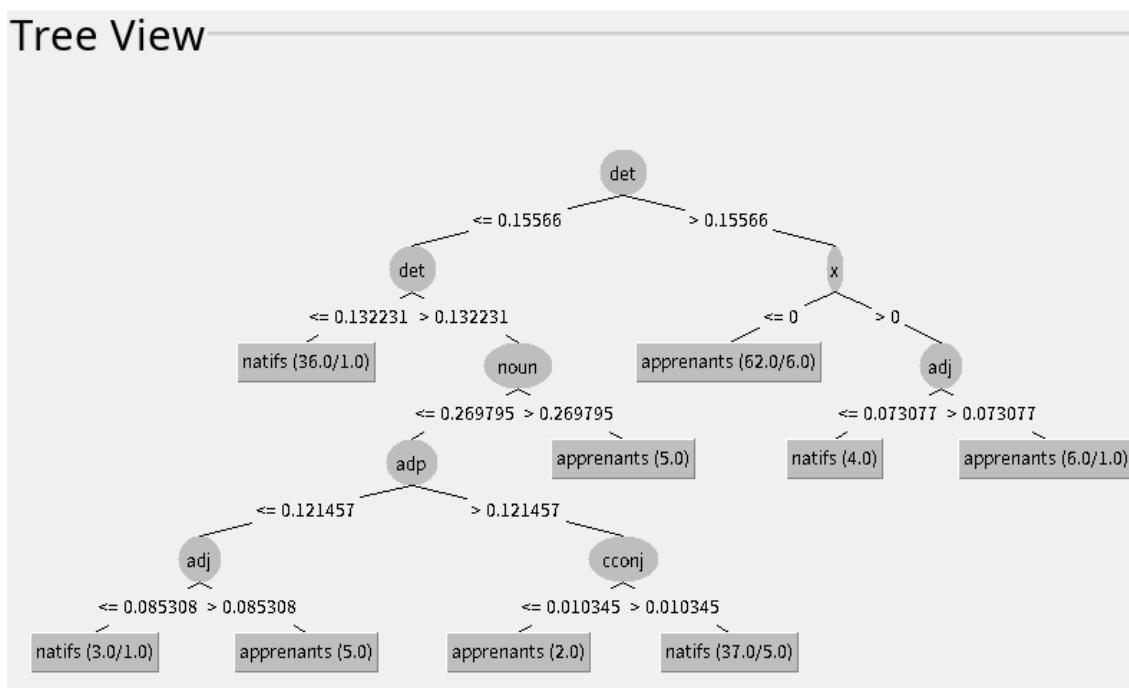
Pour SVM/SMO, les résultats sont assez proches entre les occurrences et les fréquences. En revanche, le score obtenu avec la représentation booléenne est plus faible.

Pour J48, les écarts entre les trois représentations sont plus importants. Le score est faible avec les valeurs booléennes, augmente avec les occurrences, puis atteint son niveau le plus élevé avec les fréquences.

Ces observations montrent que les trois classifieurs ne réagissent pas de la même manière aux représentations des données. Naive Bayes semble mieux exploiter les occurrences, SVM/SMO reste relativement stable entre occurrences et fréquences, tandis que J48 est plus sensible au mode de représentation et profite particulièrement des fréquences. Le meilleur résultat global est obtenu lorsque le classifieur J48 est associé à la représentation par fréquences.

4.1.3. Analyse de l'Arbre de Décision J48

Comme la représentation par fréquences avec J48 donne le meilleur score parmi les résultats initiaux, il nous a semblé utile d'analyser plus précisément le modèle produit par ce classifieur. Contrairement à Naive Bayes ou à SVM/SMO, J48 fournit un arbre de décision lisible, ce qui permet d'observer directement les catégories POS utilisées pour la classification.



arbre de décision de J48 par fréquence

Dans cet arbre, le premier nœud est **det**, avec un seuil de 0,15566. Lorsque la fréquence de **det** est supérieure à ce seuil, l'arbre se dirige vers la partie droite. Dans cette partie, une grande partie des textes est classée comme **apprenants**. Lorsque la fréquence de **det** est inférieure ou égale à 0,15566, l'arbre se dirige vers la partie gauche et continue avec d'autres nœuds.

Dans la partie gauche, **det** apparaît une deuxième fois avec un seuil de 0,132231. Lorsque la fréquence de **det** est inférieure ou égale à ce seuil, les textes sont classés comme **natifs**. Lorsque la fréquence est supérieure à ce seuil, l'arbre utilise ensuite le nœud **noun**.

À partir du nœud **noun**, plusieurs autres catégories interviennent dans les niveaux suivants, notamment **adp**, **adj** et **cconj**. Selon les seuils de ces catégories, les textes sont ensuite orientés vers la classe **natifs** ou vers la classe **apprenants**.

Ces observations montrent que J48 ne s'appuie pas sur une seule catégorie POS, mais sur une combinaison de plusieurs fréquences. Cependant, la fréquence de **det** occupe une place centrale, puisqu'elle apparaît comme nœud racine et intervient aussi dans le niveau suivant. Les autres catégories, comme **noun**, **adp**, **adj** et **cconj**, servent ensuite à affiner la classification. Cette analyse permet donc de mieux comprendre pourquoi la représentation par fréquences fonctionne bien avec J48 : elle donne au modèle des seuils de proportions exploitables pour distinguer les deux classes, tout en restant limitée au comportement observé sur notre corpus.

4.2. Optimisation par les n-grammes de POS

N-gram	Algorithmes	Accuracy(%)	Kappa	F-Measure
Bigram	NaiveBayes	77.5	0.55	0.775
	SVM(SMO)	85	0.70	0.849
	J48	65	0.28	0.631
Trigram	NaiveBayes	77.5	0.55	0.773
	SVM(SMO)	90	0.80	0.900
	J48	57.5	0.15	0.575

D'après les résultats, les bigrammes donnent de bons scores pour Naive Bayes et SVM/SMO. Naive Bayes atteint 77,5 % d'accuracy, avec une F-mesure de 0,775, tandis que SVM/SMO obtient 85 % d'accuracy et une F-mesure de 0,849. J48 obtient un résultat plus faible, avec 65 % d'accuracy et une F-mesure de 0,631.

Avec les trigrammes, Naive Bayes garde le même score d'accuracy, soit 77,5 %, avec une F-mesure très proche de celle obtenue avec les bigrammes. SVM/SMO améliore encore ses résultats et atteint 90 % d'accuracy, avec un Kappa de 0,80 et une F-mesure de 0,900. En revanche, J48 baisse avec les trigrammes et obtient seulement 57,5 % d'accuracy.

Si l'on compare les bigrammes et les trigrammes, les résultats sont donc assez différents selon les classifieurs. Naive Bayes reste stable entre les deux types de n-grammes, SVM/SMO progresse avec les trigrammes, tandis que J48 obtient de meilleurs résultats avec les bigrammes qu'avec les trigrammes.

Ces observations montrent que l'ajout des n-grammes améliore nettement les résultats par rapport aux expériences initiales, surtout pour SVM. Le meilleur résultat global est obtenu avec les trigrammes et SVM, ce qui suggère que l'ordre local des catégories POS apporte une information utile pour distinguer les deux classes. Cependant, cette optimisation ne profite pas à tous les classifieurs de la même manière : elle améliore fortement SVM, reste stable pour Naive Bayes, mais semble moins adaptée à J48, surtout avec les trigrammes.

5. Bilan

Dans ce projet, nous avons cherché à distinguer automatiquement des productions écrites par des francophones natifs et par des apprenants de français à partir de catégories morphosyntaxiques. L'objectif était de comparer plusieurs classifieurs tout en observant si ce type de représentation pouvait fournir des indices utiles pour la classification.

Les premières expériences, fondées sur les catégories POS prises séparément, montrent que le choix de la représentation des données joue un rôle important. La représentation booléenne donne les résultats les plus faibles, tandis que les occurrences et les fréquences sont plus efficaces. Le meilleur résultat initial est obtenu avec J48 et la représentation par fréquences, avec 75 % d'accuracy. L'analyse de l'arbre de décision montre que ce classifieur s'appuie notamment sur la fréquence de certaines catégories, comme *det*, *noun*, *adj* ou *cconj*, pour distinguer les deux classes.

L'ajout des n-grammes de POS a permis d'améliorer les résultats. Les bigrammes et les trigrammes prennent en compte l'ordre local des catégories morphosyntaxiques, ce qui apporte une information supplémentaire par rapport aux catégories prises séparément. Le meilleur résultat global est obtenu avec les trigrammes et SVM/SMO, avec 90 % d'accuracy et une F-mesure de 0,900. Cela montre que les enchaînements de catégories POS sont particulièrement utiles pour notre tâche.

Les résultats montrent aussi que tous les classifieurs ne réagissent pas de la même manière aux représentations des données. Naive Bayes reste relativement stable avec les n-grammes, SVM/SMO profite fortement de cette optimisation, tandis que J48 fonctionne mieux avec les fréquences simples qu'avec les trigrammes. Il est donc important de comparer à la fois les algorithmes et les modes de représentation.

Les catégories morphosyntaxiques permettent donc de distinguer, dans une certaine mesure, les productions de natifs et celles d'apprenants. Toutefois, notre travail présente plusieurs limites. D'abord, le corpus reste relativement réduit, avec 100 textes par classe. Ensuite, les résultats peuvent dépendre du choix des textes retenus, notamment du type de

production et du profil des apprenants. Enfin, l'étiquetage morphosyntaxique automatique peut contenir des erreurs, surtout dans les productions d'apprenants, où certaines formes peuvent être moins bien analysées.

Pour améliorer ce travail, il serait intéressant d'élargir le corpus, de tester d'autres tailles de n-grammes, ou encore de comparer les résultats obtenus avec d'autres types de représentations, par exemple une représentation lexicale ou une combinaison entre traits lexicaux et traits morphosyntaxiques. Cela permettrait de mieux mesurer l'apport spécifique des catégories POS dans la distinction entre productions de natifs et d'apprenants.

6. Références

1. corpus

<https://www.ortolang.fr/market/corpora/ceaal2>

2. weka

<https://ml.cms.waikato.ac.nz/weka>

3. plateforme de ressource

<https://www.ortolang.fr/fr/accueil/>

7. Annexes

- **corpus**
 - [corpus brut](#)
 - [corpus pos](#)
- **scripts**
 - [construire_corpus_natifs.py](#)
 - [construire_corpus_apprenants.py](#)
 - [vectorisation.py](#)
 - [vectorisation_ngram.py](#)
 - [transformation_pos.py](#)
 - vous trouverez l'instruction d'usage des scripts [ici](#)
- [resultats](#)
- [arff](#)